Guía simple del proyecto

Qué tenemos, qué pidió el profesor, qué hacer y qué no

Ruteo resiliente de UAV para el muestreo de contaminación en la Bahía de Quintero

Algoritmos de Ruteo y Redes Resilientes (CIT3435) · Grupo 09 · Fase 2

Cómo leer esta guía

Está escrita para que se entienda sin saber nada de antemano. Cada palabra técnica se explica la primera vez que aparece, y casi todo viene con un ejemplo de la vida diaria.

Si tienes poco tiempo, lee la página 2 (la idea en un minuto) y la página final (la lista de verificación). Si tienes más tiempo, léela entera en orden: cada sección se apoya en la anterior.

Advertencia honesta

Esta guía se basa en la instrucción 8 del profesor, que es la única que conozco textualmente, y en el informe y la presentación del grupo. Si la pauta completa de la fase 2 pide algo más, eso manda. Revísenla junto con esta guía.

1. La idea en un minuto

Si solo lees una página, que sea esta.

Lo esencial

- **Lo que el profesor pidió para la fase 2** es, en ruteo, una sola cosa: una ruta calculada con la función **pgr_dijkstra** de la base de datos, usando **solo la distancia** como costo, y que se vea en el sitio web. Esa ruta es a propósito "tonta": no mira incendios, ni población, ni nada. Es el **peor caso**.
- **Los datos de las APIs** (metadata y amenazas) en la fase 2 solo se **descargan, se guardan en la base y se muestran en el mapa**. Todavía no cambian ninguna ruta.
- **Usar esos datos para hacer una ruta más inteligente** y compararla contra el Dijkstra "tonto" es la **fase 3**.
- **Nosotros nos adelantamos**: ya hicimos el algoritmo inteligente. Eso no es malo, pero mezclamos lo obligatorio con lo adelantado, y en el apuro quedaron varias cosas enredadas o difíciles de defender.
- **El plan**: primero dejar impecable lo obligatorio. Después ordenar lo adelantado, presentarlo como "avance de la fase 3" y corregir sus puntos débiles. Antes de presentar, preguntarle al profesor si quiere ver ya ese avance.

La metáfora que usaremos en toda la guía

Piensa en un **repartidor en bicicleta** que tiene que llevar un paquete desde una bodega (el aeródromo) hasta varias casas (las estaciones de medición). Tiene un GPS.

- La fase 2 es como enseñarle al GPS el mapa de la ciudad, anotar dónde hay obras, tacos y lluvia, y mostrarle al cliente **la ruta más corta**, aunque pase por las obras.
- La fase 3 es hacer un GPS **más astuto**, que use la información de obras y tacos para elegir una ruta un poco más larga pero mucho más segura, y **demostrar** que es mejor que la ruta más corta.

2. El problema, contado como un cuento

Hay una zona industrial (Quintero, Ventanas y Puchuncaví) donde el aire está contaminado. El Estado tiene estaciones que miden el aire, pero **solo a ras de suelo**. El humo de las industrias sube y se mueve con el viento, así que sería útil mandar un **dron** a medir en altura.

El dron sale del **Aeródromo de Quintero** y tiene que llegar a **8 estaciones de medición** (estaciones SINCA). En el camino pueden aparecer cosas que hagan fallar el vuelo:

Peligro	Por qué afecta al dron	De dónde sacamos el dato
Incendio	El calor y el humo pueden derribarlo o dañarlo.	NASA FIRMS (satélites que detectan fuego)
Zonas muy pobladas	Si cae, puede herir a alguien; la norma DAN 151 restringe sobrevolarlas.	INE, Censo (manzanas con habitantes)
Sin señal	Si se pierde el enlace, no se puede controlar el dron.	Antenas de OpenStreetMap
Viento fuerte	Lo desvía o lo hace caer.	Open-Meteo
Lluvia	Puede mojar los motores y los sensores.	MET Norway
Sismo	Discutible para un dron en vuelo (ver sección 7).	EMSC / CSN

La pregunta del proyecto es: **¿cómo elegir la ruta del dron para que la misión no falle, sin alargar demasiado el vuelo?**

3. Diccionario para bebés

Estas son todas las palabras raras del proyecto, explicadas una por una. Vuelve a esta página cada vez que te pierdas.

Palabra	Qué significa, en simple
Grafo	Un dibujo de puntos unidos por líneas. Es la forma en que el computador “ve” el mapa.
Nodo	Un punto del grafo. Tenemos 237 (el aeródromo, las estaciones y otros puntos).
Arista	Una línea que une dos nodos: un tramo por donde el dron puede volar. Tenemos 633.
Costo	Lo que “cuesta” recorrer una arista. Puede ser solo la distancia, o la distancia más un castigo por peligro.
Dijkstra	El algoritmo clásico que encuentra el camino más barato entre dos puntos. Lo que dé depende de qué costo le pongas.
pgRouting / pgr_dijkstra	Una extensión de la base de datos PostgreSQL que trae Dijkstra ya programado. **pgr_dijkstra** es la función que lo ejecuta dentro de la base.
ETL	Extraer (bajar datos de las APIs), Transformar (limpiarlos y darles formato) y Cargar (guardarlos en la base).
Metadata	Datos que describen el territorio: población, antenas, relieve, calidad del aire.
Amenaza	Algo que puede hacer fallar el vuelo: incendio, sismo, viento, lluvia.
Ruta de referencia o “peor caso”	La ruta más corta, que no mira ningún peligro. Es la vara de medir.
Algoritmo propio	Nuestra forma más astuta de elegir rutas, que sí mira los peligros.
k = 2	Calcular 2 rutas: una principal y una de respaldo.
Supervivencia S	La probabilidad de recorrer un tramo sin que nada falle. Va de 0 % a 100 %.
Resiliencia R	La probabilidad de que **al menos una** de las dos rutas llegue completa.
ΔR (delta R)	La resta “R de nuestro algoritmo menos R de Dijkstra”. Si es positiva, ganamos.
λ (lambda)	Qué tan peligroso es un tramo **de verdad**: probabilidad de falla por kilómetro. Sirve para **evaluar** rutas.
α (alfa)	Cuánto **miedo** le tiene nuestro algoritmo a cada peligro. Sirve para **elegir** rutas.
β (beta)	La propuesta nueva de esta guía: una sola perilla que dice cuánto estás dispuesto a alargar el vuelo para ganar seguridad.

4. Qué pidió el profesor, frase por frase

La instrucción 8 (textual)

“El sitio deberá mostrar una ruta generada con pgr_dijkstra, utilizando la longitud de la ruta como costo, que muestre a modo de ejemplo una ruta que sea una posible solución a su problema planteado. Esta ruta será el peor caso que podría resolver su problemática, ya que no considerará la metadata ni las posibles amenazas (eso será parte de la fase 3).”

Frase	Qué quiere decir	Qué tenemos que tener
“El sitio deberá mostrar…”	La ruta tiene que verse en la página web, no solo existir en el código.	Una capa en el mapa con esa ruta.
“…generada con pgr_dijkstra…”	Calculada por la base de datos con pgRouting, no con un Dijkstra hecho a mano en Python o JavaScript.	Una consulta SQL que llame a pgr_dijkstra.
“…la longitud de la ruta como costo…”	El costo de cada tramo es solo su largo en metros.	cost = length_m, en metros reales.
“…a modo de ejemplo… una posible solución…”	Basta con un ejemplo (por ejemplo, aeródromo → una estación). Mostrar las 8 es un plus.	Al menos una ruta; ideal, poder elegir el destino.
“…el peor caso…”	Es la ruta más ingenua posible. Sirve de vara para medir lo que venga después.	Etiquetarla así en el sitio.
“…no considerará la metadata ni las amenazas…”	Esta ruta **no** debe mirar ni población, ni fuego, ni nada.	Que la consulta solo use la tabla de aristas y su largo.
“…(eso será parte de la fase 3)”	Usar metadata y amenazas para rutear es trabajo de la próxima fase.	Lo nuestro “inteligente” se presenta como adelanto.

Traducción a una frase

Para esta fase, el profesor quiere ver que **saben montar la base de datos con datos reales y calcular la ruta más corta con pgRouting**. Nada más en ruteo. Lo demás es extra.

5. Hasta dónde llegar: fase 2 contra fase 3

Esta tabla responde la duda “¿hasta qué punto llegar?”. Lo de la columna del medio es lo que **tiene** que estar ahora. Lo de la derecha es lo que viene después.

Tema	Fase 2 (ahora)	Fase 3 (después)
Base de datos	PostgreSQL + PostGIS + pgRouting en Docker, con tablas y datos cargados.	Se reutiliza.
APIs de metadata	Descargar, limpiar, guardar en la base y mostrar como capa en el mapa.	Usarlas para calcular el costo de las aristas.
APIs de amenazas	Igual: descargar, guardar y mostrar.	Usarlas para que la ruta las esquive.
Ruteo	**Una ruta pgr_dijkstra con costo = distancia.**	Algoritmo propio que use metadata y amenazas.
Comparación	No se pide.	Algoritmo propio contra Dijkstra, con una métrica.
Métrica de resiliencia R	No se pide.	Necesaria para comparar.
Sitio web	Mapa con capas y la ruta Dijkstra.	Mapa con ambas rutas, casillas de preferencias y panel de resultados.

¿Y qué hacemos con lo que ya adelantamos?

Tienen tres caminos posibles. Recomiendo el B, pero la decisión final es del profesor.

Opción	Qué significa	Riesgo
A. Quitarlo todo	Entregar solo lo de la fase 2 y guardar el algoritmo para la fase 3.	Se pierde la oportunidad de mostrar trabajo extra.
B. Mostrarlo separado (recomendado)	Entregar lo de la fase 2 impecable y, aparte, una sección “Avance de la fase 3” claramente rotulada.	Bajo, siempre que lo obligatorio esté perfecto y que el avance no tenga errores de fondo.
C. Dejarlo mezclado como está	Seguir presentando todo junto, como si fuera el centro de la fase 2.	Alto: el profesor puede sentir que no se entendió la instrucción o atacar los puntos débiles del avance.

Pregunta para hacerle al profesor (por correo, antes de presentar)

“Profesor, además de la ruta pgr_dijkstra por longitud que pide la instrucción 8, avanzamos un algoritmo que usa la metadata y las amenazas. ¿Prefiere que lo mostremos ahora como avance de la fase 3, o que nos limitemos a lo de la fase 2?”

6. Cómo estamos hoy: el semáforo

Verde es lo que está bien y hay que mantener. Amarillo es lo que hay que corregir pero no es grave. Rojo es lo que, si no se arregla, el profesor puede atacar con fuerza.

VERDE: lo que está bien

- La base de datos en Docker, con versiones fijas y un solo comando para correr todo. Muy bien.
- Ocho fuentes de datos reales descargadas automáticamente, cada una con su fuente y fecha registradas.
- Los conteos de filas cuadran entre el informe, la presentación y la base.
- Ya existe una ruta calculada con pgr_dijkstra por longitud (el panel la muestra como “rutas pgRouting”).
- El informe es honesto con sus supuestos y limitaciones.

AMARILLO: corregir

- La ruta pgr_dijkstra no está etiquetada en el sitio como “peor caso, sin metadata ni amenazas”.
- Los mapas no tienen leyenda: no se sabe qué color es qué.
- Hay nombres inconsistentes: “confiabilidad” y “resiliencia”, “adaptativa” (cuando no lo es), “rutas oficiales”, “Sombra Telecom C2”.
- El foco de incendio simulado no está guardado en la base; está escrito dentro del código.
- La web y el cálculo por lotes no usan exactamente los mismos peligros (el lote suma viento y lluvia, la web no).
- Detalles de forma: apellido del profesor mal escrito en la portada, dos láminas con el número 14, redondeos distintos para el mismo valor.

ROJO: lo que hay que entender y arreglar sí o sí

- La comparación no es justa (sección 7.2).
- El “miedo” α y el “peligro real” λ no coinciden (sección 7.3).
- La hipótesis se escribió después de ver los resultados (sección 7.4).
- Lo que se llama “Dijkstra k=2” no es lo que parece (sección 7.1).
- Hay dos programas distintos que calculan rutas (sección 7.5).

7. El caos, explicado con peras y manzanas

Esta es la parte más importante para entender por qué hay que cambiar cosas. Cada problema viene con una comparación de la vida diaria.

7.1. El “Dijkstra k=2” no es la segunda ruta más corta

Para sacar la segunda ruta de Dijkstra, el código toma la primera ruta, la “encarece” multiplicando por 4 el costo de sus tramos y vuelve a buscar. Eso es una **técnica de penalización**, la misma que usa nuestro algoritmo. O sea, los dos lados de la comparación usan el mismo método; lo único distinto es **qué costo** se usa.

Prueba: hacia la estación Sur, la segunda ruta de nuestro algoritmo (3,96 km) es más corta que la segunda ruta de “Dijkstra” (4,06 km). Si la de Dijkstra fuera la segunda más corta de verdad, eso sería imposible.

> **Cómo explicarlo bien**
>
> No estamos comparando dos algoritmos, sino **dos formas de poner precio** a los tramos: solo distancia contra distancia más peligro. Eso está bien, pero hay que decirlo así. Si se quiere la segunda ruta más corta de verdad, pgRouting tiene **pgr_KSP** (algoritmo de Yen).

7.2. La comparación no es justa

En el escenario estrella, el equipo puso un incendio **justo encima** de la ruta de Dijkstra, le dio un peligro altísimo (λ = 5, casi un muro) y nuestro algoritmo **sabía** dónde estaba el fuego. Obvio que gana.

> **La comparación con la vida diaria**
>
> Es como una carrera en que tú pusiste un bache en el carril de tu rival, tú sabías dónde estaba y él no. Ganar no demuestra que corras mejor.

La solución es “tirar dardos al mapa”: simular 1.000 incendios en lugares al azar y ver en cuántos casos gana cada uno. A eso se le llama **Monte Carlo**. Si nuestro algoritmo gana en la mayoría de los casos al azar, ahí sí hay una demostración. Además, conviene probar dos versiones: una en que el algoritmo conoce el incendio antes de salir y otra en que el incendio aparece después.

7.3. El “miedo” α y el “peligro real” λ no coinciden

Nuestro algoritmo usa α para **elegir** la ruta, y después usamos λ para **evaluarla**. Pero esos números no están de acuerdo entre sí:

	Incendio	Zona urbana	Incendio comparado con urbana
α (miedo, para elegir)	15	6	2,5 veces peor

	Incendio	Zona urbana	Incendio comparado con urbana
λ (peligro real, para evaluar)	5,0	0,08	62,5 veces peor

> **La comparación con la vida diaria**
>
> Es como si te dijeran que una calle es 62 veces más peligrosa que otra, pero tú decides tu camino como si fuera solo 2 veces peor. Vas a elegir mal en algunos casos. Por eso en Loncura y Sur nuestro algoritmo sale un poquito **peor** que Dijkstra (ΔR negativo).

La solución es elegir con el mismo número con que se evalúa. Se explica en la sección 9.

7.4. La hipótesis se escribió después de ver los resultados

La hipótesis dice “R ≥ 85 %”, “Dijkstra bajo 10 %” y “hasta 2,2 km más”. Los resultados dieron 88,6 % como mínimo, 1,5 % como máximo y 2,17 km. Calzan demasiado bien.

> **La comparación con la vida diaria**
>
> Es como apostar al resultado de un partido que ya viste. Una hipótesis se escribe **antes** de experimentar, con umbrales que vengan de afuera. Por ejemplo: “el desvío no puede superar el 20 % de la batería del dron X”, con la ficha técnica del dron citada.

7.5. Hay dos programas distintos que calculan rutas

La ruta de Dijkstra sale de pgr_dijkstra en la base, pero nuestro algoritmo usa un Dijkstra escrito a mano en Python (se ve en la lámina 13: `dijkstra_camino(adj, ...)`). Son dos calculadoras distintas.

> **La comparación con la vida diaria**
>
> Si comparas dos pesos usando dos balanzas diferentes, no sabes si la diferencia viene de los objetos o de las balanzas. Lo ideal es usar **la misma balanza** (pgr_dijkstra) para todo y solo cambiar el costo.

7.6. Otros detalles que el profesor puede preguntar

- **Tramos enteros “contaminados”:** si un tramo de 1 km toca un incendio de 180 m de radio, hoy se cuenta el kilómetro entero como peligroso. Lo correcto es contar solo los metros que quedan dentro.
- **El “punto crítico” de 450 m:** ese bloqueo existe en nuestro dibujo de la red (cada punto unido solo a sus 4 vecinos), no en el cielo real, donde un dron puede rodear el fuego. Hay que decirlo así.

- **¿Para qué dos rutas si el dron vuela una?** R significa “al momento de despegar, hay al menos una ruta libre y elijo esa”. Cambiar de ruta en pleno vuelo es otra cosa (el planificador local, que no se hizo).
- **Sismos:** un sismo de hace tres semanas no afecta a un dron volando. O se justifica (por ejemplo, daño a la zona de aterrizaje) o se saca.
- **Antenas de celular:** solo importan si el dron se controla por red celular (4G). Si no, la capa no mide lo que dice medir.

8. Qué hacer, paso a paso

Hagan los pasos en este orden. No pasen al siguiente sin terminar el anterior.

Paso 1. Dejar impecable lo obligatorio (fase 2)

Esto es lo único que **no puede** fallar.

- La ruta se calcula con **pgr_dijkstra dentro de la base**, con este costo y nada más: `cost = length_m`.
- `length_m` está en **metros reales** (calculado con `ST_Length(geom::geography)`, no en grados).
- En el sitio hay una capa llamada “Ruta de referencia: pgr_dijkstra (costo = longitud)”, con el texto: “Peor caso: no considera metadata ni amenazas (fase 3)”.
- Se puede elegir el destino entre las 8 estaciones, y se muestran la distancia y el número de tramos.
- La ruta se ve siempre, aunque todo lo demás esté apagado.
- Las distancias que muestra el sitio coinciden con la Tabla 8 del informe.

Así se ve la consulta (para que no quede duda)

```
SELECT d.seq, d.edge, d.agg_cost, e.geom
FROM pgr_dijkstra(
'SELECT id, source, target,
length_m AS cost, length_m AS reverse_cost
FROM infra_edge',
:origen, :destino, directed := false) d
LEFT JOIN infra_edge e ON e.id = d.edge
ORDER BY d.seq;
```

Fíjense en que **solo** aparece la tabla de aristas. Ni población, ni incendios, ni antenas.

Paso 2. Ordenar los datos (fase 2)

- Que cada fuente se vea como capa en el mapa, con leyenda.
- Guardar el foco de incendio simulado en la tabla de amenazas, con una marca “simulado = sí”.
- Revisar que el grafo esté todo conectado (que desde el aeródromo se pueda llegar a las 8 estaciones).
- Guardar una “foto” (snapshot) de los datos del día en que se sacaron los resultados, porque los datos en vivo cambian.
- Explicar en el informe para qué sirve cada tabla, incluidas las que todavía no se usan para rutear (calidad del aire, relieve).

Paso 3. Arreglar el informe y la presentación

- Separar claramente: “Fase 2: lo pedido” y “Avance de la fase 3”.
- Reescribir la hipótesis con umbrales que no salgan de los propios resultados.

- Decir con honestidad que la comparación actual es de **funciones de costo**, no de algoritmos distintos.
- Corregir los nombres, las leyendas, el apellido del profesor y la numeración.
- Preparar láminas de respaldo para las preguntas difíciles (α contra λ, Monte Carlo, cómo se calculan las distancias).

Paso 4. Mejorar el algoritmo propio (avance de la fase 3)

Solo si hay tiempo después de los pasos 1 a 3. Se explica en la sección siguiente.

9. Cómo debería ser nuestro algoritmo (fase 3)

9.1. La idea, sin fórmulas

El algoritmo propio **no es un Dijkstra distinto**. Es el mismo pgr_dijkstra, pero le damos un mapa de precios diferente: los tramos peligrosos “cuestan más”, así que Dijkstra los evita solo. Después buscamos una segunda ruta distinta, por si la primera falla.

Son dos ideas:

- **Idea 1: ponerle precio al peligro.** Cada tramo cuesta su distancia más un recargo por el peligro que tiene.
- **Idea 2: buscar un plan B.** Encarecemos los tramos de la primera ruta y volvemos a buscar. La segunda ruta tiende a irse por otro lado. Esa es nuestra **heurística**: no garantiza el mejor par posible, pero es simple y funciona bien.

9.2. Ponerle precio al peligro, con un ejemplo

Proponemos esta regla, que reemplaza a los α:

> **Costo del tramo = distancia en metros + β × riesgo del tramo**
>
> donde el **riesgo** es un riesgo base pequeño (λ = 0,005 por km, para todo tramo) más la suma, para cada peligro, de su λ por los kilómetros del tramo que caen dentro de ese peligro. β es una sola perilla que dice cuántos metros extra estoy dispuesto a volar para evitar una unidad de riesgo.

Ejemplo con β = 1.000 metros:

Tramo de 1 km que pasa por…	Riesgo (λ × km)	Recargo (β × riesgo)	Costo final	En palabras
Nada peligroso (solo riesgo base)	0,005 × 1 = 0,005	5 m	1.005 m	Casi igual que la distancia.
Zona urbana	(0,005 + 0,08) × 1 = 0,085	85 m	1.085 m	Un poquito más caro.
Incendio	(0,005 + 5) × 1 = 5,005	≈ 5.005 m	≈ 6.005 m	Para el algoritmo, ese kilómetro “vale” como 6.

¿Por qué esta regla es mejor que la de los α?

- Usa **el mismo λ** para elegir y para evaluar. Se acaba la contradicción de la sección 7.3.
- Con β = 0, el algoritmo da **exactamente** la ruta de Dijkstra. Eso sirve como prueba de que todo funciona.
- Con β muy grande, encuentra la ruta **más segura posible**.
- Probando varios β (0, 250, 500, 1.000, 2.000, 5.000…) se dibuja una curva que muestra cuánta seguridad se gana por cada kilómetro extra. Eso responde

directamente al objetivo “maximizar R minimizando el aumento de distancia”.

9.3. Cómo se calcula la resiliencia R, con un ejemplo

Imagina dos puentes para cruzar un río. Para llegar a ellos hay un camino común.

- El camino común llega sano con 95 % de probabilidad.
- El puente 1 resiste con 90 % y el puente 2 también con 90 %.
- La probabilidad de que se caigan **los dos** puentes es 10 % × 10 % = 1 %. Entonces al menos uno funciona con 99 %.
- R = 95 % × 99 % ≈ 94 %.

Así funciona nuestra fórmula. Y la supervivencia de un tramo se calcula con la regla “mientras más largo y más peligroso, menos probable llegar”: con λ = 5 (incendio), un kilómetro dentro del fuego deja apenas un 0,7 % de probabilidad de salir bien.

9.4. Contra qué comparar (para que sea justo)

Código	Qué es	Para qué sirve
B0	pgr_dijkstra, solo distancia, 1 ruta	**Lo que pide la instrucción 8.** El peor caso.
B1	Mismo método de 2 rutas, β = 0	Ver cuánto aporta solo el precio del peligro.
B2	pgr_KSP (Yen), 2 rutas más cortas	Responder “¿por qué no usaron Yen?”.
A1	Nuestro costo, 1 ruta	Ver cuánto aporta evitar peligros sin plan B.
A2	Nuestro costo, 2 rutas	Nuestro algoritmo completo.

Con esta tabla, cada pregunta del profesor tiene un experimento que la responde.

9.5. Los experimentos que darían una demostración de verdad

- **Escenario base:** datos reales del día, todos los métodos, las 8 estaciones.
- **Incendio simulado:** el de Planta Asfaltos, como ahora.
- **Dardos al azar (Monte Carlo):** 1.000 incendios en lugares aleatorios, con tamaños de 180, 450 y 1.000 m. Contar en qué porcentaje de casos llega el dron con cada método. Este es el experimento que más valor le da al trabajo.
- **Perillas:** probar distintos β y distintos factores de penalización (×2, ×4, ×10).

10. Qué NO hacer

Prohibido

- **No** agregarle metadata ni amenazas a la ruta pgr_dijkstra de referencia. Si se la agregan, deja de cumplir la instrucción 8.
- **No** esconder la ruta de referencia cuando el algoritmo propio está encendido.
- **No** calcular rutas en el navegador (JavaScript). Las rutas salen de la base.
- **No** repetir los mismos números (λ, α, β) en varios archivos. Van en uno solo.
- **No** llamar “KSP”, “Yen”, “adaptativo” o “geodésico” a algo que no lo es.
- **No** ajustar los parámetros para que los resultados “se vean mejor”. Si se cambia algo, se explica por qué.
- **No** decir “los resultados confirman la hipótesis”. Mejor: “son consistentes con la hipótesis bajo los supuestos del modelo”.
- **No** presentar el incendio de Planta Asfaltos como la única prueba.

11. Cómo defenderse en la presentación

Regla de oro: **reconocer una debilidad y decir cómo se resolvería vale más que defender algo indefendible**. A nivel de magíster, eso se lee como madurez.

Si el profesor pregunta…	Respondan algo así
“¿Esto es lo que pedí?”	“Sí: la ruta pgr_dijkstra por longitud está aquí, rotulada como peor caso. Lo otro es un avance de la fase 3 que mostramos aparte.”
“¿No es obvio que gana si ustedes pusieron el fuego?”	“Sí, ese escenario solo muestra el mecanismo. Para demostrarlo de verdad proponemos incendios al azar (Monte Carlo).”
“¿De dónde salen esos números?”	“Son supuestos. Por eso hicimos sensibilidad, y en la fase 3 proponemos usar un solo parámetro β derivado de λ.”
“¿Su Dijkstra k=2 es la segunda ruta más corta?”	“No; es la misma técnica de penalización con costo = distancia. La segunda más corta real se obtiene con pgr_KSP y la agregaremos como comparación.”
“¿Para qué dos rutas si el dron vuela una?”	“R es la probabilidad de que al despegar haya al menos una ruta libre. Cambiar de ruta en vuelo es el planificador local, fuera de alcance.”

12. Lista de verificación final

Marquen cada casilla antes de entregar.

Obligatorio de la fase 2

- ■ La ruta sale de **pgr_dijkstra** en la base de datos.
- ■ Su costo es **solo la longitud**, en metros reales.
- ■ **No** usa metadata ni amenazas.
- ■ Se ve en el sitio, con etiqueta de “peor caso”.
- ■ Se puede ver hacia al menos una estación SINCA (ideal: las 8).
- ■ Las 4 fuentes de metadata y las 4 de amenazas están cargadas y se ven en el mapa.
- ■ Todo corre con un comando en Docker.
- ■ Revisamos la pauta completa del profesor por si pide algo más.

Orden y consistencia

- ■ Mapas con leyenda.
- ■ Mismos nombres en el informe, la presentación y la web.
- ■ Mismos números en el informe, la presentación y la web.
- ■ El foco simulado está guardado en la base.
- ■ Hay un snapshot de los datos usados en los resultados.
- ■ Apellido del profesor y numeración de láminas corregidos.

Avance de la fase 3 (si se presenta)

- ■ Le preguntamos al profesor si quiere verlo ahora.
- ■ Está separado y rotulado como “Avance de la fase 3”.
- ■ Se explica como comparación de funciones de costo.
- ■ Hipótesis reescrita con umbrales independientes de los resultados.
- ■ Tenemos respuesta preparada para “la comparación no es justa” (Monte Carlo como siguiente paso).
- ■ Láminas de respaldo listas para las preguntas difíciles.

Si solo recuerdas una cosa

La fase 2 es **datos bien cargados + la ruta más corta con pgr_dijkstra, mostrada en el sitio**. Si eso está perfecto, la entrega cumple. Todo lo demás es para lucirse, y solo luce si está bien explicado.